FUNDACIÓN ESCUELA TECNOLOGICA DE NEIVA
JESÚS OVIEDO PÉREZ

Taller de Laboratorio N.º 3: Procesamiento Asíncrono de Eventos: API, base de datos, y cola de mensajes

Autor:
Andres Ricardo Rodriguez

Fundación Escuela Tecnológica de Neiva "FET" Jesus Oviedo Pérez

Docente: Juan Carlos Polanía Cortes

Ingeniería de Software – Optativa IV

2026

Tabla de Contenido

1. Introducción 3
2. Objetivos 3
2.1. Objetivo General 3
2.2. Objetivos Específicos 3
3. Instructivo de Reproducción 4
4. Casos Comprobados 4
5. Preguntas de Cierre 11
6. Declaración de uso de herramientas de inteligencia artificial 12

1. Introducción

En este laboratorio se integraron la API con base de datos MySQL del laboratorio 1 y la mensajería con RabbitMQ del laboratorio 2 para construir un flujo de procesamiento asíncrono de eventos. El caso de uso es el registro de un pago: la API valida los datos, guarda el pago con estado REGISTRADO, publica un mensaje con su identificador en una cola y responde de inmediato al cliente. De forma independiente, un consumidor toma el mensaje, ejecuta la acción de procesamiento, actualiza el pago a PROCESADO y deja constancia en una segunda tabla. Los cuatro servicios (MySQL, RabbitMQ, API y consumidor) se ejecutan en contenedores Docker y se levantan con un único comando.

2. Objetivos

2.1. Objetivo General

Implementar un flujo asíncrono en el que el registro de una operación y su procesamiento posterior sean independientes, y demostrar esa independencia con mediciones propias.

2.2. Objetivos Específicos

- Desarrollar una API (FastAPI) que valide, inserte el pago, publique el mensaje y responda sin esperar el procesamiento.
- Desarrollar un consumidor que procese con confirmación manual (ACK al final) y registre la acción.
- Medir tiempos de respuesta y de procesamiento, y comprobar tres escenarios: consumidor detenido, acción más lenta y fallo al procesar.

3. Instructivo de Reproducción

1. Desde la raíz, para partir de cero: *docker compose down -v* y luego *docker compose up --build -d*.
2. Verificar: *docker compose ps* (los 4 servicios en ejecución) y *docker compose logs -f consumer*.
3. Accesos: API en http://localhost:8000 (POST y GET /pagos), panel de RabbitMQ en http://localhost:15672 (guest / guest).

El máximo corresponde al primer pago, que incluye la apertura de las primeras conexiones a MySQL y RabbitMQ. Sin ese primer valor, la media de los 19 restantes es de 26,7 ms. Los tiempos de procesamiento crecen unos 4 s por pago (4,171 s, 8,037 s, 12,032 s...) porque hay un único consumidor con prefetch_count=1, que atiende los mensajes uno tras otro; por eso el último pago espera ~80 s y la API nunca lo notó.

4. Casos Comprobados

Caso 1: consumidor detenido

Procedimiento:

1. Se detuvo el consumidor:

docker compose stop consumer

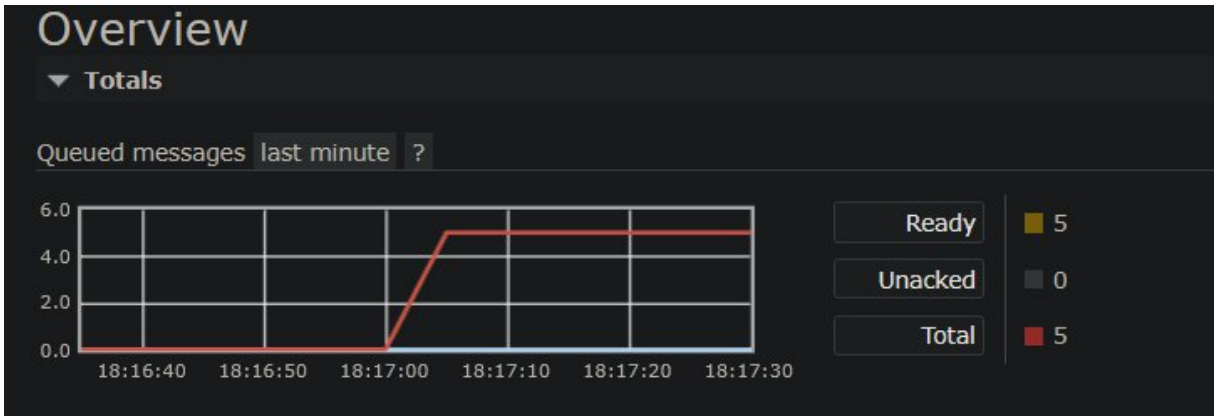
2. Se registraron cinco pagos desde PowerShell, mostrando el status y el estado de cada respuesta*:*



La API respondió status=True y estado=REGISTRADO en los cinco casos.

```
1 status=True estado=REGISTRADO
2 status=True estado=REGISTRADO
3 status=True estado=REGISTRADO
4 status=True estado=REGISTRADO
5 status=True estado=REGISTRADO
```

3. Se verificó la cola en el panel de RabbitMQ (http://localhost:15672, pestaña Queues): la cola pagos_queue mostró 5 mensajes en Ready y 0 consumidores.



4. Se consultó el estado de los pagos dentro de MySQL:

docker exec -it taller_db mysql -uuser -ppassword pagos_db

SELECT id, referencia, estado, fecha_registro FROM pagos ORDER BY id DESC LIMIT 10;

```
mysql> SELECT id, referencia, estado, fecha_registro FROM pagos ORDER BY id DESC LIMIT 10;
+----+------------+------------+-------------------------+
| id | referencia | estado     | fecha_registro          |
+----+------------+------------+-------------------------+
|  5 | CASO1-5    | REGISTRADO | 2026-09-28 23:16:58.872 |
|  4 | CASO1-4    | REGISTRADO | 2026-09-28 23:16:58.838 |
|  3 | CASO1-3    | REGISTRADO | 2026-09-28 23:16:58.804 |
|  2 | CASO1-2    | REGISTRADO | 2026-09-28 23:16:58.774 |
|  1 | CASO1-1    | REGISTRADO | 2026-09-28 23:16:58.643 |
+----+------------+------------+-------------------------+
5 rows in set (0.01 sec)
```

Los cinco pagos (CASO1-1 a CASO1-5) aparecieron en estado REGISTRADO.

5. Se reinició el consumidor:

docker compose start consumer

docker logs taller_consumer --tail 30

El log mostró los cinco pagos tomados de la cola y finalizados, con su hora de inicio y de fin.

6. Se volvió a consultar la base de datos con las mismas consultas y además la tabla de procesamientos:

SELECT id, referencia, estado, fecha_registro FROM pagos ORDER BY id DESC LIMIT 10;

*SELECT * FROM procesamientos ORDER BY id DESC LIMIT 10;*

```
mysql> SELECT id, referencia, estado, fecha_registro FROM pagos ORDER BY id DESC LIMIT 10;
+----+------------+-----------+-------------------------+
| id | referencia | estado    | fecha_registro          |
+----+------------+-----------+-------------------------+
|  5 | CASO1-5    | PROCESADO | 2026-09-28 23:16:58.872 |
|  4 | CASO1-4    | PROCESADO | 2026-09-28 23:16:58.838 |
|  3 | CASO1-3    | PROCESADO | 2026-09-28 23:16:58.804 |
|  2 | CASO1-2    | PROCESADO | 2026-09-28 23:16:58.774 |
|  1 | CASO1-1    | PROCESADO | 2026-09-28 23:16:58.643 |
+----+------------+-----------+-------------------------+
5 rows in set (0.00 sec)

mysql> SELECT * FROM procesamientos ORDER BY id DESC LIMIT 10;
+----+---------+-------------------------+----------------------------------------+
| id | pago_id | fecha_procesamiento     | resultado                              |
+----+---------+-------------------------+----------------------------------------+
|  5 |       5 | 2026-09-28 23:19:56.983 | Comprobante generado exitosamente en 4s |
|  4 |       4 | 2026-09-28 23:19:52.928 | Comprobante generado exitosamente en 4s |
|  3 |       3 | 2026-09-28 23:19:48.876 | Comprobante generado exitosamente en 4s |
|  2 |       2 | 2026-09-28 23:19:44.854 | Comprobante generado exitosamente en 4s |
|  1 |       1 | 2026-09-28 23:19:40.836 | Comprobante generado exitosamente en 4s |
+----+---------+-------------------------+----------------------------------------+
5 rows in set (0.00 sec)
```

Los cinco pagos pasaron a PROCESADO y cada uno tiene su registro en procesamientos. La cola volvió a 0 mensajes.

Resultado: los pagos se registraron a las 23:16:58 y se procesaron entre las 23:19:40 y las 23:19:65, con unos 4 s entre uno y otro. Ningún mensaje se perdió mientras el consumidor estuvo detenido. Eso se debe a que la cola es durable, los mensajes se publican como persistentes y el consumidor confirma (ACK) solo después de terminar de procesar.

Caso 2: acción más lenta

Procedimiento:

1. Se reinició el consumidor con un delay de 15 s (el valor entra por la variable de entorno PROCESSING_DELAY del compose):

$env:PROCESSING_DELAY=15; docker compose up -d consumer

docker logs -f taller_consumer

En el log se comprobó que cada pago tardaba unos 15 s entre "Tomado de la cola" y "Finalizado".

2. Se registraron cinco pagos (LENTO-1 a LENTO-5) midiendo el tiempo de respuesta de la API:

```
PS C:\WINDOWS\System32>    $tiempos = 1..5 | ForEach-Object {
>>      $body = @{ referencia = "LENTO-$_"; valor = 125000; medio = "transferencia" } | ConvertTo-Json
>>      $t = Measure-Command {
>>        $r = Invoke-RestMethod -Uri http://localhost:8000/pagos -Method Post -ContentType "application/json" -Body $bo
dy
>>      }
>>      "{0}  {1} ms  status={2}" -f $_, [math]::Round($t.TotalMilliseconds, 1), $r.status | Write-Host
>>      $t.TotalMilliseconds
>>    }
1  153,5 ms  status=True
2  38,8 ms  status=True
3  74,8 ms  status=True
4  73,2 ms  status=True
5  73,1 ms  status=True
PS C:\WINDOWS\System32>    "Media: {0} ms  Máximo: {1} ms" -f [math]::Round(($tiempos | Measure-Object -Average).Average
, 1), [math]::Round(($tiempos | Measure-Object -Maximum).Maximum, 1)
Media: 82,7 ms  Máximo: 153,5 ms
PS C:\WINDOWS\System32>
```

Los cinco respondieron status=True, con tiempos de 153,5; 38,8; 74,8; 73,2 y 73,1 ms. La media fue de 82,7 ms y el máximo de 153,5 ms (el primer pago).

3. Se esperó a que los cinco quedaran procesados (unos 75 s) y se consultó la base de datos:

docker exec -it taller_db mysql -uuser -ppassword pagos_db

SELECT p.id, p.referencia, p.estado, TIMESTAMPDIFF(MICROSECOND, p.fecha_registro, pr.fecha_procesamiento)/1000 AS ms FROM pagos p LEFT JOIN procesamientos pr ON pr.pago_id = p.id WHERE p.referencia LIKE 'LENTO-%' ORDER BY p.id DESC LIMIT 5;

Los cinco pagos quedaron en PROCESADO. El tiempo entre el registro y el procesamiento fue de 15,052 s, 30,033 s, 44,990 s, 59,967 s y 74,955 s: cada pago se procesó unos 15 s después del anterior, porque hay un único consumidor que atiende los mensajes en orden.

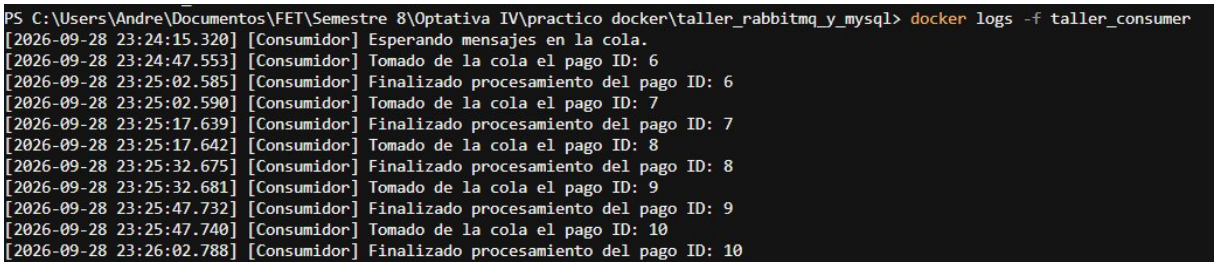
4. Se restauró el delay original:

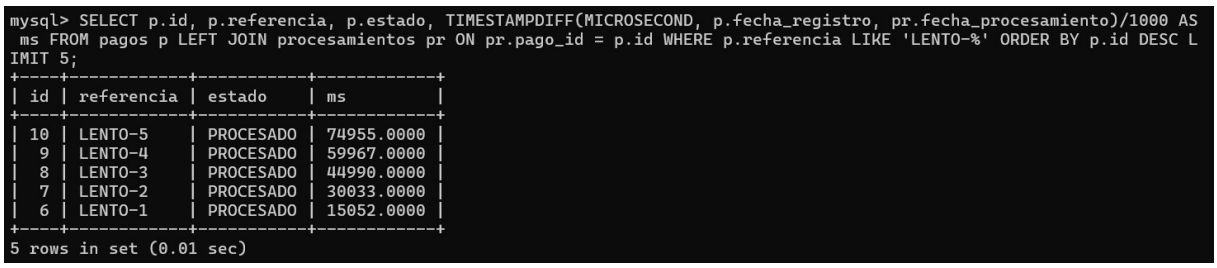
Remove-Item Env:PROCESSING_DELAY

docker compose up -d consumer

	Delay 4 s	Delay 15 s
Media de respuesta de la API	37,7 ms (20 pagos**)**	82,7 ms (5 pagos)
Máximo de respuesta de la API	247,8 ms	153,5 ms
Registro → procesado, primer pago	4,171 s	15,052 s
Registro → procesado, último pago	80,226 s (de 20)	74,955 s (de 5)

Análisis: la acción del consumidor pasó de 4 a 15 s, es decir, 11 000 ms más por pago, y el tiempo de registro a procesado se movió en la misma proporción. En cambio, la respuesta de la API se mantuvo en decenas de milisegundos: la media pasó de 37,7 a 82,7 ms, y en ningún caso superó los 155 ms en esta prueba. Si la API esperara al consumidor, habría tardado más de 15 000 ms por pago. El aumento en la media es pequeño frente a esa diferencia y se atribuye a la variabilidad del entorno (Docker sobre Windows, primeras conexiones) y al tamaño reducido de la muestra (5 pagos contra 20 de la base), no a una dependencia entre la API y el consumidor. Esto es coherente con el diseño: la API responde apenas guarda el pago y publica el mensaje.





Caso 3: fallo al procesar

Objetivo: comprobar que si el consumidor falla mientras procesa un mensaje, el mensaje no se pierde y el pago no queda marcado como PROCESADO.

Procedimiento:

1. Se activó un fallo simulado en el consumidor mediante la variable de entorno FAIL_ON_PROCESS, que lanza una excepción durante el procesamiento, después del delay y antes de actualizar el pago:

$env:FAIL_ON_PROCESS="true"; docker compose up -d consumer

docker logs -f taller_consumer

2. Se registró un pago (FALLO-1):

```
PS C:\WINDOWS\System32>    $body = @{ referencia = "FALLO-1"; valor = 125000; medio = "transferencia" } | ConvertTo-Json

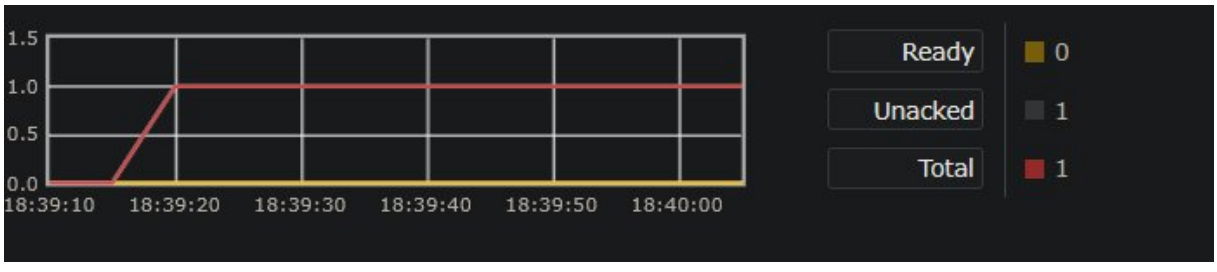
PS C:\WINDOWS\System32>    Invoke-RestMethod -Uri http://localhost:8000/pagos -Method Post -ContentType "application/jso
n" -Body $body

status message         data
------ -------         ----
  True Pago registrado @{id=11; estado=REGISTRADO}
```

La API respondió status=True y estado REGISTRADO: desde su punto de vista el pago quedó guardado y encolado, sin saber que el consumidor iba a fallar.

3. En el log del consumidor apareció "Tomado de la cola el pago ID ..." seguido de [ERROR] Fallo al procesar pago ID ...: Fallo simulado durante el procesamiento. Como el mensaje vuelve a la cola, el ciclo se repite cada ~7 s (4 s de procesamiento más 3 s de espera antes de devolverlo).

4. Se consultó la cola en RabbitMQ (http://localhost:15672, pestaña Queues):




```
PS C:\Users\Andre\Documentos\FET\Semestre 8\Optativa IV\practico docker\taller_rabbitmq_y_mysql> docker logs -f taller_consumer
[2026-09-28 23:35:53.406] [Consumidor] Esperando mensajes en la cola.
[2026-09-28 23:39:10.775] [Consumidor] Tomado de la cola el pago ID: 11
[2026-09-28 23:39:14.798] [ERROR] Fallo al procesar pago ID 11: Fallo simulado durante el procesamiento
[2026-09-28 23:39:17.802] [Consumidor] Tomado de la cola el pago ID: 11
[2026-09-28 23:39:21.820] [ERROR] Fallo al procesar pago ID 11: Fallo simulado durante el procesamiento
[2026-09-28 23:39:24.826] [Consumidor] Tomado de la cola el pago ID: 11
[2026-09-28 23:39:28.849] [ERROR] Fallo al procesar pago ID 11: Fallo simulado durante el procesamiento
[2026-09-28 23:39:31.851] [Consumidor] Tomado de la cola el pago ID: 11
[2026-09-28 23:39:35.862] [ERROR] Fallo al procesar pago ID 11: Fallo simulado durante el procesamiento
```

La cola pagos_queue mantuvo 1 mensaje (casi todo el tiempo en Unacked, mientras el consumidor lo intenta procesar) y nunca bajó a 0.

5. Se consultó la base de datos:

SELECT id, referencia, estado FROM pagos WHERE referencia = 'FALLO-1';

*SELECT * FROM procesamientos WHERE pago_id = (SELECT id FROM pagos WHERE referencia = 'FALLO-1');*

```
mysql>    SELECT id, referencia, estado FROM pagos WHERE referencia = 'FALLO-1';
+----+------------+------------+
| id | referencia | estado     |
+----+------------+------------+
| 11 | FALLO-1    | REGISTRADO |
+----+------------+------------+
1 row in set (0.02 sec)

mysql>    SELECT * FROM procesamientos WHERE pago_id = (SELECT id FROM pagos WHERE referencia = 'FALLO-1')
Empty set (0.02 sec)
```

El pago siguió en estado REGISTRADO y la segunda consulta devolvió un conjunto vacío: no se creó ningún registro de procesamiento.

6. Se desactivó el fallo:

Tras reiniciar, el consumidor tomó el mensaje pendiente y lo procesó con éxito. Se repitieron las dos consultas SQL: el pago pasó a PROCESADO, apareció su registro en procesamientos y la cola volvió a 0.

7. Se limpió la variable:

Remove-Item Env:FAIL_ON_PROCESS

```
mysql>    SELECT * FROM procesamientos WHERE pago_id = (SELECT id FROM pagos WHERE referencia = 'FALLO-1');
+----+---------+-------------------------+-----------------------------------------+
| id | pago_id | fecha_procesamiento     | resultado                               |
+----+---------+-------------------------+-----------------------------------------+
| 11 |      11 | 2026-09-28 23:42:25.203 | Comprobante generado exitosamente en 4s |
+----+---------+-------------------------+-----------------------------------------+
1 row in set (0.00 sec)

mysql>    SELECT id, referencia, estado FROM pagos WHERE referencia = 'FALLO-1';
+----+------------+-----------+
| id | referencia | estado    |
+----+------------+-----------+
| 11 | FALLO-1    | PROCESADO |
+----+------------+-----------+
1 row in set (0.00 sec)
```

Análisis: el mensaje no se perdió porque el consumidor envía la confirmación (ACK) solo al final, después de actualizar el pago y registrar el procesamiento. Cuando ocurre un error, se hace rollback de la transacción, así que el pago sigue en REGISTRADO, y el mensaje se devuelve a la cola con nack y requeue, por lo que se reintenta hasta que el procesamiento tenga éxito. La API no se enteró del fallo: su respuesta ya se había entregado al cliente antes de que el consumidor tomara el mensaje.

5. Preguntas de Cierre

1. ¿Cambió el tiempo de respuesta de la API al pasar la acción del consumidor de cinco a quince segundos? ¿Por qué?

No de forma significativa. Con el consumidor en 4 s, la API respondió con una media de 37,7 ms (20 pagos, máximo 247,8 ms); con 15 s, la media fue de 82,7 ms (5 pagos, máximo 153,5 ms). La media subió unas decenas de milisegundos, pero la acción del consumidor aumentó 11 000 ms por pago, y el tiempo entre el registro y el procesamiento del primer pago pasó de 4,171 s a 15,052 s. Si la API esperara al consumidor, habría tardado más de 15 000 ms por petición; respondió en menos de 155 ms en todos los casos de esta prueba. La diferencia entre las medias se atribuye a la variabilidad del entorno y a la muestra pequeña (5 pagos contra 20), y no a una dependencia entre la API y el consumidor. La razón de fondo es que la API responde apenas guarda el pago y publica el mensaje; lo que ocurre después es un evento independiente.

2. Con el consumidor detenido, la API respondió que el pago fue registrado. ¿Qué le está prometiendo esa respuesta al cliente y qué no le está prometiendo?

Le promete que el pago quedó guardado en la base de datos con estado REGISTRADO y que el mensaje quedó en una cola durable con publicación persistente, de modo que se procesará cuando haya un consumidor disponible. No le promete que el pago ya esté procesado, ni cuándo lo estará, ni que el consumidor esté funcionando. En el caso 1, los cinco pagos se registraron a las 23:16:58 y quedaron en REGISTRADO hasta que se levantó el consumidor; se procesaron entre 162,193 s y 178,111 s después del registro (casi tres minutos), sin que la respuesta que recibió el cliente cambiara en nada.

3. Si este sistema tuviera un plazo declarado, ¿sobre qué parte del flujo se podría declarar y sobre cuál no? Justifique con sus mediciones.

Se podría declarar sobre la parte síncrona: validar, insertar, publicar y responder. Su tiempo fue de decenas de milisegundos en todas las pruebas (media de 37,7 ms y máximo de 247,8 ms con 20 pagos) y no dependió de la duración de la acción del consumidor. No se podría declarar sobre el procesamiento completo hasta PROCESADO, porque su duración depende de factores que la API no controla:

- Cuántos pagos hay en cola. Con 20 pagos seguidos, el primero se procesó a los 4,171 s y el último a los 80,226 s, unos 4 s más por cada pago pendiente.
- La duración de la acción. Con 15 s por pago, los cinco pagos tardaron entre 15,052 s y 74,955 s.

- Que el consumidor esté activo. Con el consumidor detenido, el tiempo de procesamiento fue de 162,193 s a 178,111 s, y no tiene cota: depende de cuánto tiempo siga apagado.

Para declarar un plazo también sobre el procesamiento habría que limitar la carga que entra a la cola, aumentar el número de consumidores y vigilar que siempre haya uno activo.

6. Declaración de uso de herramientas de inteligencia artificial

Se utilizó Claude (Anthropic) como apoyo técnico y de redacción: revisión del docker-compose.yml, de la API, del consumidor y del script SQL, orientación para medir los tiempos y provocar los tres casos de prueba, y ayuda en la redacción de procedimientos, análisis y respuestas de cierre. Con esa asistencia se añadieron al proyecto la variable FAIL_ON_PROCESS y el delay configurable del consumidor, las fechas con precisión de milisegundos (TIMESTAMP(3)). Una primera medición se descartó por errores de PowerShell y se repitió. Todas las pruebas, mediciones, capturas y la colección de Postman fueron ejecutadas y verificadas por el autor en su propio entorno.